



ADMINISTRACIÓN DE SERVIDORES

DOCUMENTACIÓN SAMBA Y FTP

PROYECTO GRUPAL

José Luis Olmo Barberá
Victor Manuel Vidal Molina
Alvaro Griñón Martínez

CURSO 2024/25

Índice

1. Explicación contenido añadido a la infraestructura	2
1.1. Contenedor Samba-FTP en Development	3
1.1.1. Código del contenedor samba_ftp en el docker-compose.yml	3
1.1.2. Especificaciones técnicas	3
1.1.3. Código del dockerfile usado para crear una imagen personalizada . .	4
1.1.4. Código del script 'entrypoint.sh' usado en el Dockerfile	5
1.1.5. Explicación dockerfile y entrypoint	6
1.1.6. Código del archivo 'smb.conf'	7
1.1.7. Explicación contenido 'smb.conf'	7
1.1.8. Código del archivo 'vsftpd.conf'	9
1.1.9. Explicación contenido 'vsftpd.conf'	10
1.1.10. Comprobaciones dentro del contenedor:	11
1.1.11. Acceso a Samba desde el host:	12
1.1.12. Acceso a FTP desde el host:	17
1.2. Contenedor Punto de Acceso en Production	18
1.2.1. Código del contenedor sftp_gateway en el docker-compose.yml: . .	18
1.2.2. Especificaciones técnicas:	18
1.2.3. Código del dockerfile usado para crear una imagen personalizada: .	20
1.2.4. Código del script 'entrypoint.sh' usado en el Dockerfile	21
1.2.5. Explicación dockerfile y entrypoint:	22
1.2.6. Código del archivo 'sshd_config'	23
1.2.7. Explicación contenido 'sshd_config'	23
1.2.8. Comprobaciones dentro del contenedor:	24
1.2.9. Acceso al servidor sftp desde el host:	25

1. Explicación contenido añadido a la infraestructura

Durante esta fase de implementación, se han incorporado dos nuevos contenedores a la infraestructura anterior, cada uno con una configuración específica. A continuación se detallan las características y configuraciones implementadas en cada uno:

■ Contenedor en Development:

• Servidor Samba:

- Directorios 'desarrollo', 'revision' y 'publico' con sus cinco subdirectorios cada uno sobre los que trabajan los empleados.
- Empleados no tienen acceso a archivos de otros empleados.
- Cuando un empleado genera un archivo lo colocan en 'revision' en su subdirectorio correspondiente a la espera que el supervisor de el visto bueno.
- Solo existe un supervisor, con acceso a 'revision' y cuando da el visto bueno mueve el archivo deseado al directorio 'publico' en su subdirectorio correspondiente, en el que el supervisor es el unico con permisos de escritura, cualquier usuario solo tiene lectura.

• Servidor FTP:

- Solo se puede acceder como anonymous, sin pedir contraseña.
- Ofrece el servicio de acceder a la documentación localizada en el directorio "publico".
- El usuario anónimo solo tiene permisos de lectura.
- Protocolo FTP básico, sin cifrar datos ni comunicaciones.

■ Contenedor en Production:

• Servidor Punto de Acceso:

- Punto de acceso a la red local de la empresa.
- Bloquear cualquier petición a los puertos FTP.
- Ofrecer acceso a los clientes al servidor FTP instalado en el primer punto.
- Una sola conexión simultánea.
- Se permite fijar los puertos para FTP si así se consigue aumentar la seguridad.

1.1. Contenedor Samba-FTP en Development

1.1.1 Código del contenedor samba_ftp en el docker-compose.yml

```
# Bloque Samba-FTP y servidor acceso publico
samba_ftp:
  build: ./development/samba_ftp
  container_name: samba_ftp
  restart: always
  cap_add:
    - NET_ADMIN
  dns:
    - 172.20.0.6
  networks:
    development_net:
      ipv4_address: 172.40.0.10
    host_access_net:
      ipv4_address: 172.40.1.10
  ports:
    - "21:21"
    - "139:139"
    - "445:445"
    - "40000-40010:40000-40010"
  volumes:
    - ./development/samba_ftp/data:/data
```

Figura 1: Contenedor samba_ftp.

1.1.2 Especificaciones técnicas

- Nombre del contenedor: samba_ftp.
- IP asignada: 172.40.0.10.
- IP asignada para acceder desde el host: 172.40.1.10.
- DNS: 172.20.0.6.
- Puertos: '21:21', '139:139', '445:445', '40000-40010:40000-40010'.
- Redes: development_net que lo conecta a la infraestructura y host_access_net permite conectar el host con el contenedor.

1.1.3 Código del dockerfile usado para crear una imagen personalizada

```
# samba_ftp /Dockerfile
FROM debian:bullseye

ENV DEBIAN_FRONTEND=noninteractive

RUN apt update && apt install -y samba vsftpd sudo nano net-tools iputils-ping dnsutils iproute2

RUN useradd -m john && echo 'john:john' | chpasswd && usermod -s /bin/bash john

RUN mkdir -p /var/run/vsftpd/empty && chmod 755 /var/run/vsftpd/empty

COPY conf/smb.conf /etc/samba/smb.conf
COPY conf/vsftpd.conf /etc/vsftpd.conf
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh

EXPOSE 21 139 445 40000-40010

CMD ["/bin/bash", "/entrypoint.sh"]
```

Figura 2: Dockerfile para crear imagen personalizada de samba_ftp.

1.1.4 Código del script 'entrypoint.sh' usado en el Dockerfile

```
#!/bin/bash

ip route del default 2>/dev/null

iface_dev=$(ip -o -4 addr show | grep 'inet 172\.40\.0\.' | awk '{print $2}' | head -n1)

if [[ -n "$iface_dev" ]]; then
    echo "Asignando gateway 172.40.0.1 a traves de $iface_dev"
    ip route add default via 172.40.0.1 dev "$iface_dev"
fi

echo "nameserver 172.20.0.6" > /etc/resolv.conf

for i in {1..5}; do
    useradd -M -s /sbin/nologin empleado$i
    echo -e "password\npassword" | smbpasswd -a -s empleado$i
done

useradd -M -s /sbin/nologin revisor
echo -e "password\npassword" | smbpasswd -a -s revisor

for i in {1..5}; do
    mkdir -p /data/desarrollo/SW$i
    mkdir -p /data/revision/SW$i
    mkdir -p /data/publico/SW$i

    chown -R empleado$i:empleado$i /data/desarrollo/SW$i
    chmod -R 770 /data/desarrollo/SW$i

    chown -R empleado$i:revisor /data/revision/SW$i
    chmod -R 770 /data/revision/SW$i

    chown -R revisor:revisor /data/publico/SW$i
    chmod -R 755 /data/publico/SW$i
done

/usr/sbin/vsftpd /etc/vsftpd.conf &
/usr/sbin/smbd -F --no-process-group
```

Figura 3: Contenido script ejecutado en el dockerfile de samba_ftp.

1.1.5 Explicación dockerfile y entryptpoint

■ Dockerfile:

- Actualizamos e instalamos los paquetes necesarios en el contenedor.
- Creamos el usuario John y damos permisos, que aunque no se especificaba nada, al igual que en la práctica anterior lo hemos añadido a todos los contenedores.
- Creamos los directorios con permisos adecuados necesarios para vsftpd.
- Copiamos los archivos de configuración de samba y ftp al contenedor.
- Copiamos el script de arranque y le damos permiso de ejecución.
- Declaramos los puertos que utilizará el contenedor.
- Por último, ejecutamos el script 'entrypoint.sh' como comando principal del contenedor.

■ Entrypoint:

- Eliminamos la ruta por defecto.
- Detectamos la interfaz de development_net y la establecemos como gateway.
- Establecemos el servidor DNS.
- Creamos los usuarios empleados del 1 al 5 para samba, sin acceso al shell(nologin) y con contraseña 'password'.
- Creamos el usuario revisor de la misma forma.
- Creamos los subdirectorios SW1 a SW5 en las tres áreas: desarrollo, publico, revision.
- Establecemos los permisos de los empleados y del revisor con respecto al guión.
- Iniciamos el servidor FTP con la configuración definida, ejecutándolo en segundo plano.
- Finalmente, iniciamos el servidor Samba en primer plano.

1.1.6 Código del archivo 'smb.conf'

```
[global]
workgroup = WORKGROUP
security = user
map to guest = Bad User
server string = Samba Server
log file = /var/log/samba/log.%m
max log size = 50
dns proxy = no

[desarrollo]
path = /data/desarrollo
browseable = yes
read only = no
create mask = 0770
directory mask = 0770
valid users = empleado1 empleado2 empleado3 empleado4 empleado5
force user = %U

[revision]
path = /data/revision
browseable = yes
read only = no
create mask = 0770
directory mask = 0770
valid users = revisor
force group = revisor

[revision1]
path = /data/revision/SW1
browseable = yes
read only = no
create mask = 0770
directory mask = 0770
valid users = empleado1 revisor
force group = revisor

[revision2]
path = /data/revision/SW2
browseable = yes
read only = no
create mask = 0770
directory mask = 0770
valid users = empleado2 revisor
force group = revisor

[revision3]
path = /data/revision/SW3
browseable = yes
read only = no
create mask = 0770
directory mask = 0770
valid users = empleado3 revisor
force group = revisor

[revision4]
path = /data/revision/SW4
browseable = yes
read only = no
create mask = 0770
directory mask = 0770
valid users = empleado4 revisor
force group = revisor

[revision5]
path = /data/revision/SW5
browseable = yes
read only = no
create mask = 0770
directory mask = 0770
valid users = empleado5 revisor
force group = revisor

[publico]
path = /data/publico
browseable = yes
read only = yes
guest ok = yes
write list = revisor
```

Figura 4: Contenido del archivo 'smb.conf'.

1.1.7 Explicación contenido 'smb.conf'

A continuación vamos a explicar cada uno de los bloques línea por línea:

■ **Global:**

- Se define el grupo de trabajo para compartir los recursos en red, en este caso WORKGROUP.
- El sistema requiere autenticación con usuario y contraseña.
- Si un usuario se autentica con credenciales inválidas, se redirige al usuario guest si está permitido.

- Nombre del servidor Samba en la red.
- Define la ubicación del archivo de registro.
- Limite máximo del tamaño del archivo de log.
- Evita que Samba intente resolver nombres de red mediante DNS.

■ **Desarrollo:**

- Ruta del sistema de archivos que se compartirán.
- Permite que este recurso se muestre al explorar el servidor en red.
- Permite que los usuarios escriban en el recurso compartido.
- Definimos los permisos por defecto al crear archivos o carpetas: lectura, escritura y ejecución para el propietario y grupo, ningún permiso para el resto.
- Solo esos usuarios pueden acceder a los recursos.
- Fuerza que los archivos creados tengan como propietario el mismo usuario que accede.

■ **Revision/ Revision1-5:**

- Ruta del sistema de archivos que se compartirán.
- Se puede explorar y los usuarios tienen permiso de escritura.
- Permisos por defectos de los archivos y carpetas: lectura y escritura para el usuario y su grupo, ningún permiso para el resto.
- Solo esos usuarios pueden acceder a los recursos.
- Todos los archivos creados en este recursos pertenecerán al grupo revisor, lo que garantiza que el revisor pueda acceder a ellos.
- Como se puede observar, hemos creado varios bloques revision, uno en el que el revisor puede acceder a todo el directorio y en los bloques revisionX solo se tiene acceso al subdirectorio correspondiente por parte del revisor y el empleadoX, impidiendo acceder a los empleados a los subdirectorios de otros empleados.

■ **Publico:**

- Ruta del sistema de archivos que se compartirán.
- Se puede explorar pero no modificar (excepto usuarios autorizados).
- Permite el acceso sin autenticar como invitado.
- Solo revisor tiene permisos de escritura, aunque el recurso sea de solo lectura para el resto.

1.1.8 Código del archivo 'vsftpd.conf'

```
listen=YES
listen_ipv6=NO

# Solo acceso anonimo, sin usuario local
anonymous_enable=YES
local_enable=NO
write_enable=NO

# Raiz para el usuario anonymous
anon_root=/data/publico
anon_world_readable_only=YES
anon_umask=022

# Seguridad minima: solo FTP basico
chroot_local_user=NO
allow_writeable_chroot=NO

# No cifrado
ssl_enable=NO

# Modo pasivo
pasv_enable=YES
pasv_min_port=40000
pasv_max_port=40010

# Otros ajustes
dirmessage_enable=NO
use_localtime=YES
xferlog_enable=NO
```

Figura 5: Contenido del archivo 'vsftpd.conf'.

1.1.9 Explicación contenido 'vsftpd.conf'

A continuación, explicaremos el funcionamiento del código línea a línea:

- Habilitamos el modo escucha en IPv4 y deshabilitamos la escucha en IPv6.
- Permitimos el acceso anónimo al servidor FTP como se solicita en el guión.
- Impedimos el acceso mediante usuarios del sistema, solo pueden acceder usuarios anónimos.
- Desactiva la escritura en el servidor FTP. Los usuarios solo pueden leer o descargar los archivos.
- Definimos el directorio raíz para el usuario anónimo. Aquí es donde estará la documentación accesible.
- Restringimos a los usuarios anónimos a leer únicamente archivos con permisos globales de lectura.
- Establecemos la umask, permitiendo lectura pública y escritura solo para el propietario.
- Desactivamos el chroot para usuarios locales, por seguridad.
- Impedimos entornos chroot que sean escribibles, de nuevo por seguridad.
- Desactivamos el cifrado SSL, forzando el uso de FTP básico tal como se indica en el guión.
- Activamos el modo pasivo de FTP, especificando un rango limitado de puertos.
- Desactivamos los mensajes al acceder a los directorios.
- Hacemos que el servidor utilice la hora local del sistema.
- Por último, desactivamos el registro de transferencias, evitando la generación de archivos de logs innecesarios.

1.1.10 Comprobaciones dentro del contenedor:

Accedemos al contenedor y hacemos una serie de comprobaciones como asegurarnos que el gateway y el dns son los correctos y para comprobar la conectividad haremos varios pings, por ejemplo probamos a hacer ping al dns, que es un contenedor de la subred services y también haremos ping a un contenedor de la propia red development, por ejemplo al servidor ssh, vemos que ambos pings llegan correctamente, sin embargo el último ping, con destino al servidor Nginx, ubicado en la red production no llega, esto es lo correcto según los guiones de las prácticas anteriores, todo lo descrito se ve en la siguiente imagen:

```
john@DebianDocker:~/Escritorio/AS/PRACTICAS$ docker exec -it samba_ftp bash
root@7f4c831ff59e:/# ip route
default via 172.40.0.1 dev eth0
172.40.0.0/24 dev eth0 proto kernel scope link src 172.40.0.10
172.40.1.0/24 dev eth1 proto kernel scope link src 172.40.1.10
root@7f4c831ff59e:/# cat /etc/resolv.conf
nameserver 172.20.0.6
root@7f4c831ff59e:/# ping 172.20.0.6
PING 172.20.0.6 (172.20.0.6) 56(84) bytes of data.
64 bytes from 172.20.0.6: icmp_seq=1 ttl=63 time=34.2 ms
^C
--- 172.20.0.6 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 34.166/34.166/34.166/0.000 ms
root@7f4c831ff59e:/# ping 172.40.0.9
PING 172.40.0.9 (172.40.0.9) 56(84) bytes of data.
64 bytes from 172.40.0.9: icmp_seq=1 ttl=64 time=1.46 ms
64 bytes from 172.40.0.9: icmp_seq=2 ttl=64 time=0.553 ms
^C
--- 172.40.0.9 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 0.553/1.006/1.459/0.453 ms
root@7f4c831ff59e:/# ping 172.30.0.5
PING 172.30.0.5 (172.30.0.5) 56(84) bytes of data.
^C
--- 172.30.0.5 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3078ms

root@7f4c831ff59e:/# █
```

Figura 6: Comprobaciones dentro del contenedor samba_ftp.

1.1.11 Acceso a Samba desde el host:

Desde la terminal de nuestro host, introducimos los siguientes comandos:

```
sudo ip link add macvlan0 link ens20 type macvlan mode bridge
```

```
sudo ip addr add 172.40.1.100/24 dev macvlan0
```

```
sudo ip link set macvlan0 up
```

Con estos comandos haremos que nuestro host tenga una IP con la cual podremos hacer conexión.

A continuación, veremos como se conectan los diferentes empleados, el revisor y el usuario anónimo, además, mostraremos que puede hacer cada usuario una vez inicia la conexión:

- **EmpleadoX:** En este caso usaremos el empleado1 como ejemplo, siendo igual para todos los empleados unicamente cambiando el número del empleado en el comando de acceso a Samba.

- Acceso al directorio 'desarrollo':

Introducimos desde la terminal de nuestro host el siguiente comando:

```
smbclient //172.40.1.10/desarrollo -U empleado1
```

Introducimos la clave 'password' y habremos accedido al servidor Samba, como podemos observar, el usuario empleado1 solo puede acceder al subdirectorio SW1, el correspondiente a su usuario, siendo imposible acceder a los subdirectorios de otros usuarios. Por último, vemos como el empleado1 sube un archivo al directorio SW1.

```
john@DebianDocker:~/Escritorio/AS/PRACTICAS$ smbclient //172.40.1.10/desarrollo -U empleado1
Password for [WORKGROUP\empleado1]:
Try "help" to get a list of possible commands.
smb: \> cd SW2
cd \SW2\ : NT_STATUS_ACCESS_DENIED
smb: \> cd SW3
cd \SW3\ : NT_STATUS_ACCESS_DENIED
smb: \> cd SW4
cd \SW4\ : NT_STATUS_ACCESS_DENIED
smb: \> cd SW5
cd \SW5\ : NT_STATUS_ACCESS_DENIED
smb: \> cd SW1
smb: \SW1\> put /etc/hostname prueba.txt
putting file /etc/hostname as \SW1\prueba.txt (0,0 kb/s) (average 0,0 kb/s)
smb: \SW1\> ls
.                D           0   Sat May 10 18:04:21 2025
..               D           0   Wed May  7 00:07:26 2025
prueba.txt       A          13   Sat May 10 18:04:22 2025
test.txt         A          13   Sat May 10 04:37:59 2025

19480400 blocks of size 1024. 5753008 blocks available
smb: \SW1\>
```

Figura 7: Empleado1 solo puede acceder a SW1 y sube un archivo.

- Acceso al directorio 'revision':

Introducimos desde la terminal de nuestro host el siguiente comando:

```
smbclient //172.40.1.10/revision1 -U empleado1
```

Usamos la contraseña 'password' para acceder y como podemos comprobar con la siguiente imagen, el empleado1 solo tiene acceso al subdirectorio correspondiente a su usuario ya que así está definido el bloque 'revision1' en el archivo 'smb.conf' y no se le permite acceder a otros, en este subdirectorio el empleado1 puede subir archivos definitivos. Además, vemos como al principio empleado1 intenta acceder al directorio global 'revision' y no puede ya que no tiene permiso de acceso.

```
john@DebianDocker:~/Escritorio/AS/PRACTICAS$ smbclient //172.40.1.10/revision -U empleado1
Password for [WORKGROUP\empleado1]:
tree connect failed: NT_STATUS_ACCESS_DENIED
john@DebianDocker:~/Escritorio/AS/PRACTICAS$ smbclient //172.40.1.10/revision1 -U empleado1
Password for [WORKGROUP\empleado1]:
Try "help" to get a list of possible commands.
smb: \> ls
.                               D          0  Sat May 10 18:06:39 2025
..                              D          0  Wed May  7 00:07:43 2025
19480400 blocks of size 1024. 5752684 blocks available
smb: \> put /etc/hostname prueba.txt
putting file /etc/hostname as \prueba.txt (0,6 kb/s) (average 0,6 kb/s)
smb: \> ls
.                               D          0  Sat May 10 18:07:54 2025
..                              D          0  Wed May  7 00:07:43 2025
prueba.txt                      A         13  Sat May 10 18:07:54 2025
19480400 blocks of size 1024. 5752680 blocks available
smb: \> █
```

Figura 8: Empleado1 accede a directorio 'revision' y sube un archivo definitivo.

- Acceso al directorio 'publico':

Introducimos desde la terminal de nuestro host el siguiente comando:

```
smbclient //172.40.1.10/publico -U empleado1
```

Introducimos 'password' como contraseña y mostramos todo el directorio, el empleado1 tiene acceso a cualquier subdirectorio pero solo tiene permisos de lectura, lo que impide escribir en cualquiera de los subdirectorios, en este directorio solo tiene permiso de escritura el revisor.

```
John@DebianDocker:~/Escritorio/AS/PRACTICA5$ smbclient //172.40.1.10/publico -U empleado1
Password for [WORKGROUP\empleado1]:
Try "help" to get a list of possible commands.
smb: \> ls
.                D            0 Wed May  7 00:07:51 2025
..               D            0 Wed May  7 00:07:51 2025
SW5              D            0 Wed May  7 00:07:51 2025
SW3              D            0 Wed May  7 00:07:51 2025
SW1              D            0 Sat May 10 18:13:40 2025
SW4              D            0 Wed May  7 00:07:51 2025
SW2              D            0 Wed May  7 00:07:51 2025
19480400 blocks of size 1024. 5751488 blocks available
smb: \> cd SW1
smb: \SW1\> ls
.                D            0 Sat May 10 18:13:40 2025
..               D            0 Wed May  7 00:07:51 2025
prueba.txt       A           13 Sat May 10 18:13:40 2025
19480400 blocks of size 1024. 5751488 blocks available
smb: \SW1\> get prueba.txt
getting file \SW1\prueba.txt of size 13 as prueba.txt (0,8 KiloBytes/sec) (average 0,8 KiloBytes/sec)
smb: \SW1\> put /etc/hostname pruebaaa.txt
NT_STATUS_ACCESS_DENIED opening remote file \SW1\pruebaaa.txt
smb: \SW1\> cd ..
smb: \> cd SW2
smb: \SW2\> cd ..
smb: \> cd SW3
smb: \SW3\> cd ..
smb: \> cd SW4
smb: \SW4\> cd ..
smb: \> cd SW5
smb: \SW5\> ls
.                D            0 Wed May  7 00:07:51 2025
..               D            0 Wed May  7 00:07:51 2025
19480400 blocks of size 1024. 5751488 blocks available
smb: \SW5\> cd ..
smb: \>
```

Figura 9: Empleado1 accede al directorio 'publico' donde puede leer todo el directorio.

■ Revisor:

- Acceso al directorio 'desarrollo':

Introducimos desde la terminal de nuestro host el siguiente comando:

```
smbclient //172.40.1.10/desarrollo -U revisor
```

El revisor no tiene acceso al directorio 'desarrollo' según el guión por tanto dará error.

```
john@DebianDocker:~/Escritorio/AS/PRACTICAS$ smbclient //172.40.1.10/desarrollo -U revisor
Password for [WORKGROUP\revisor]:
tree connect failed: NT_STATUS_ACCESS_DENIED
john@DebianDocker:~/Escritorio/AS/PRACTICAS$
```

Figura 10: Revisor no puede acceder al directorio 'desarrollo'.

- Acceso al directorio 'revision':

Introducimos desde la terminal de nuestro host el siguiente comando:

```
smbclient //172.40.1.10/revision -U revisor
```

Usamos la contraseña 'password' para acceder y como podemos comprobar con la siguiente imagen, el revisor tiene acceso a todos los subdirectorios, accedemos a SW1, donde anteriormente subimos un archivo de prueba con el empleado1, descargaremos dicho archivo para darle el visto bueno y posteriormente cargarlo en el subdirectorio correspondiente dentro del directorio 'publico'.

```
john@DebianDocker:~/Escritorio/AS/PRACTICAS$ smbclient //172.40.1.10/revision -U revisor
Password for [WORKGROUP\revisor]:
Try "help" to get a list of possible commands.
smb: \> ls
.                D           0   Wed May  7 00:07:43 2025
..               D           0   Wed May  7 00:07:51 2025
SW5              D           0   Wed May  7 00:07:43 2025
SW3              D           0   Wed May  7 00:07:43 2025
SW1              D           0   Sat May 10 18:07:54 2025
SW4              D           0   Wed May  7 00:07:43 2025
SW2              D           0   Sat May 10 04:20:22 2025

19480400 blocks of size 1024. 5744636 blocks available
smb: \> cd SW1
smb: \SW1\> ls
.                D           0   Sat May 10 18:07:54 2025
..               D           0   Wed May  7 00:07:43 2025
prueba.txt       A          13   Sat May 10 18:07:54 2025

19480400 blocks of size 1024. 5744636 blocks available
smb: \SW1\> get prueba.txt
getting file \SW1\prueba.txt of size 13 as prueba.txt (0,2 KiloBytes/sec) (average 0,2 KiloBytes/sec)
smb: \SW1\>
```

Figura 11: Revisor descarga el archivo del empleado1.

- Acceso al directorio 'publico':

Introducimos desde la terminal de nuestro host el siguiente comando:

```
smbclient //172.40.1.10/publico -U revisor
```

Introducimos 'password' como contraseña y mostramos todo el directorio, el revisor tiene acceso a cualquier subdirectorio pero con permisos de lectura y escritura, vemos como el revisor publica el archivo validado anteriormente al empleado1 en el subdirectorio SW1.

```
John@DebianDocker:~/Escritorio/AS/PRACTICAS$ smbclient //172.40.1.10/publico -U revisor
Password for [WORKGROUP\revisor]:
Try "help" to get a list of possible commands.
smb: \> ls
.                D            0   Wed May  7 00:07:51 2025
..               D            0   Wed May  7 00:07:51 2025
SW5              D            0   Wed May  7 00:07:51 2025
SW3              D            0   Wed May  7 00:07:51 2025
SW1              D            0   Sat May 10 20:02:40 2025
SW4              D            0   Wed May  7 00:07:51 2025
SW2              D            0   Wed May  7 00:07:51 2025
19480400 blocks of size 1024. 5744640 blocks available
smb: \> cd SW1
smb: \SW1\> ls
.                D            0   Sat May 10 20:02:40 2025
..               D            0   Wed May  7 00:07:51 2025
19480400 blocks of size 1024. 5744640 blocks available
smb: \SW1\> put prueba.txt
putting file prueba.txt as \SW1\prueba.txt (0,6 kb/s) (average 0,6 kb/s)
smb: \SW1\> ls
.                D            0   Sat May 10 20:03:23 2025
..               D            0   Wed May  7 00:07:51 2025
prueba.txt       A           13   Sat May 10 20:03:23 2025
19480400 blocks of size 1024. 5744636 blocks available
smb: \SW1\> █
```

Figura 12: Revisor publica el archivo del empleado1 tras el visto bueno.

1.1.12 Acceso a FTP desde el host:

Desde la terminal de nuestro host, utilizaremos el siguiente comando:

```
ftp 172.40.1.10
```

Seguidamente introducimos el usuario 'anonymous' y cuando pida contraseña pulsamos 'enter', es decir, no introducimos contraseña, esto nos dará acceso al directorio 'publico', a continuación, mostramos el directorio y accedemos al subdirectorio SW1 para descargar el archivo publicado anteriormente. Además, intentamos eliminar dicho archivo lo que provocará un error, esto es correcto ya que el usuario 'anonymous' solo tiene permiso de lectura.

Todo lo detallado anteriormente se puede ver en la siguiente imagen:

```
john@ubuntu:~/Escritorio/AS/PRACTICAS$ ftp 172.40.1.10
Connected to 172.40.1.10.
220 (vsFTPd 3.0.3)
Name (172.40.1.10:john): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||40004|)
150 Here comes the directory listing.
drwxr-xr-x  2 1006      1006          4096 May 10 18:03 SW1
drwxr-xr-x  2 1006      1006          4096 May 06 22:07 SW2
drwxr-xr-x  2 1006      1006          4096 May 06 22:07 SW3
drwxr-xr-x  2 1006      1006          4096 May 06 22:07 SW4
drwxr-xr-x  2 1006      1006          4096 May 06 22:07 SW5
226 Directory send OK.
ftp> cd SW1
250 Directory successfully changed.
ftp> ls
229 Entering Extended Passive Mode (|||40000|)
150 Here comes the directory listing.
-rwxr--r--  1 1006      1006           13 May 10 18:03 prueba.txt
226 Directory send OK.
ftp> get prueba.txt
local: prueba.txt remote: prueba.txt
229 Entering Extended Passive Mode (|||40005|)
150 Opening BINARY mode data connection for prueba.txt (13 bytes).
100% |*****| 13 75.12 KiB/s 00:00 ETA
226 Transfer complete.
13 bytes received in 00:00 (11.01 KiB/s)
ftp> delete prueba.txt
550 Permission denied.
ftp>
```

Figura 13: Anonymous hace login FTP y descarga el archivo publicado por el revisor.

1.2. Contenedor Punto de Acceso en Production

1.2.1 Código del contenedor sftp_gateway en el docker-compose.yml:

```
sftp_gateway:
  build: ./production/sftp_gateway
  container_name: sftp_gateway
  restart: always
  privileged: true
  cap_add:
    - NET_ADMIN
  devices:
    - /dev/fuse
  security_opt:
    - apparmor:unconfined
  dns:
    - 172.20.0.6
  networks:
    production_net:
      ipv4_address: 172.30.0.10
    development_net:
      ipv4_address: 172.40.0.15
    bridge_local: {}
  ports:
    - "2224:22"
  volumes:
    - ./production/sftp_gateway/sftp_data:/home/cliente/sftp_data
  depends_on:
    - samba_ftp
```

Figura 14: Contenedor sftp_gateway.

1.2.2 Especificaciones técnicas:

- Nombre del contenedor: sftp_gateway
- Modo privilegiado le da permisos extendidos dentro del sistema.
- Capacidades adicionales: NET_ADMIN es necesario para modificar las rutas.
- Dispositivos y seguridad: /dev/fuse permite sistemas de archivos montables en espacio de usuario y apparmor:unconfined elimina restricciones.
- IP asignada: 172.20.0.10.

- DNS: 172.20.0.6.
- Redes: production_net que lo conecta a la infraestructura, development_net permite conectar el contenedor sftp_gateway con el servidor ftp y bridge_local para conectar el host a la infraestructura.
- Puertos de acceso: "2224:22".
- Dependencias: El contenedor no se iniciará hasta que samba_ftp esté listo.

1.2.3 Código del dockerfile usado para crear una imagen personalizada:

```
# sftp_gateway /Dockerfile
FROM debian:bullseye

ENV DEBIAN_FRONTEND=noninteractive

RUN apt update && apt install -y \
    openssh-server \
    curlftpfs \
    sudo \
    nano \
    netcat-openbsd \
    net-tools \
    iputils-ping \
    dnsutils \
    iproute2

RUN useradd -m john && echo "john:john" | chpasswd && adduser john sudo

RUN useradd -m -d /home/cliente -s /bin/bash cliente && \
    echo "cliente:password" | chpasswd && \
    mkdir -p /home/cliente/sftp_data && \
    chown cliente:cliente /home/cliente/sftp_data

RUN mkdir /var/run/sshd

COPY sshd_config /etc/ssh/sshd_config
COPY entrypoint.sh /entrypoint.sh
COPY usuarios.txt /usuarios.txt
RUN chmod +x /entrypoint.sh

EXPOSE 22

CMD ["/entrypoint.sh"]
```

Figura 15: Dockerfile para crear imagen personalizada de sftp_gateway.

1.2.4 Código del script 'entrypoint.sh' usado en el Dockerfile

```
#!/bin/bash
set -e

# Configurar gateway para producción
ip route del default 2>/dev/null
iface_prod=$(ip -o -4 addr show | grep 172\.30\. | awk '{print $2}' | head -n1)
if [[ -n "$iface_prod" ]]; then
    echo "Asignando gateway 172.30.0.1 a través de $iface_prod"
    ip route add default via 172.30.0.1 dev "$iface_prod"
fi

# DNS interno
echo "nameserver 172.20.0.6" > /etc/resolv.conf

# Crear punto de montaje
mkdir -p /mnt/ftp_real

# Variables de conexión FTP anónimo
FTP_USER="anonymous"
FTP_PASS=""
FTP_IP="172.40.0.10"

# Esperar que puerto 21 esté disponible
echo "Esperando que el puerto FTP 21 esté disponible en $FTP_IP..."
while ! nc -z -w 1 $FTP_IP 21; do
    echo "Puerto 21 aún no disponible, esperando 5s..."
    sleep 5
done

# Intentar montar FTP remoto
for i in {1..5}; do
    echo "Intentando montar FTP remoto (intento $i)..."
    if curlftpfs ${FTP_USER}:${FTP_PASS}@$FTP_IP /mnt/ftp_real -o allow_other,disable_epsv
    then
        echo "Montaje FTP exitoso"
        break
    else
        echo "No se pudo montar el FTP. Esperando 10s..."
        sleep 10
    fi
done

# Crear enlace simbólico para el usuario sftp
ln -sfn /mnt/ftp_real /home/cliente/sftp_data

# Ejecutar SSH
/usr/sbin/sshd -D
```

Figura 16: Contenido script ejecutado en el dockerfile de sftp_gateway.

1.2.5 Explicación dockerfile y entrypoint:

■ Dockerfile:

- Usamos la imagen Debian Bullseye como imagen base.
- Evitamos prompts interactivos al instalar paquetes con apt.
- Actualizamos e instalamos los paquetes necesarios en el contenedor.
- Creamos el usuario John y damos permisos, que aunque no se especificaba nada, al igual que en la práctica anterior lo hemos añadido a todos los contenedores.
- Creamos el usuario cliente y directorio, habilitamos la shell, contraseña 'password' y creamos el directorio de trabajo de SFTP, por último cambiamos la propiedad de dicho directorio al usuario nuevo.
- Creamos el directorio para el servicio SSH, esto es necesario para que el servicio SSH funcione correctamente.
- Copiamos los archivos desde el host al contenedor y otorgamos los permisos necesarios para su ejecución.
- Exponemos el puerto 22 del contenedor para conexiones SSH/SFTP.
- El contenedor ejecuta el script 'entrypoint.sh' cuando se inicie.

■ Entrypoint:

- Si algún comando falla el script se detiene inmediatamente.
- Eliminamos la ruta por defecto si es que existe y asignamos la nueva ruta.
- Asignamos la IP del DNS interno, en este caso la IP es la asignada al contenedor bind9 que actúa como DNS.
- Creamos el directorio donde se montará el contenido del servidor FTP remoto.
- Definimos las variables para FTP anónimo.
- Nos mantenemos a la espera hasta que el puerto 21 esté disponible reiniciando cada 5 segundos hasta que esté accesible.
- Montamos el servidor FTP remoto, realizamos 5 intentos para montarlo usando curlftpfs, si tiene éxito se rompe el bucle, si falla lo reintentará dentro de 10 segundos.
- Creamos un enlace simbólico al directorio FTP. Esto hace que el usuario vea el contenido del FTP remoto como si fuera su propio directorio.
- Iniciamos el servidor SSH en primer plano.

1.2.6 Código del archivo 'sshd_config'

```
Port 22
Protocol 2
PermitRootLogin no
PasswordAuthentication yes
Subsystem sftp internal-sftp

Match Group sftputers
    ChrootDirectory /home/%u
    ForceCommand internal-sftp
    AllowTcpForwarding no
    X11Forwarding no
```

Figura 17: Contenido del archivo 'sshd_config'.

1.2.7 Explicación contenido 'sshd_config'

A continuación, explicaremos el funcionamiento del código línea a línea:

- El servidor SSH escuchará en el puerto 22, que es el puerto estándar para conexiones SSH/SFTP.
- Se indica que solo se permitirá el uso del protocolo SSH versión 2.
- Prohíbe que el usuario root inicie sesión directamente por SSH/SFTP.
- Permite autenticación por contraseña. Esto debe estar activado si los usuarios se conectan usando contraseña y no llaves SSH.
- Define que el servicio SFTP se ejecutará mediante el binario internal-sftp que viene con el servidor SSH.
- Aplicamos reglas solo para los usuarios que pertenezcan al grupo sftputers.
- Restringe a cada usuario a su propio directorio (/home/nombre_usuario). Esto evita que puedan navegar fuera de su carpeta personal. %u se sustituye automáticamente por el nombre del usuario.
- Obliga a que estos usuarios solo puedan usar SFTP, incluso si intentan abrir una sesión SSH. Así no pueden acceder a la línea de comandos.
- Desactiva el reenvío de puertos TCP para estos usuarios, una medida adicional de seguridad.
- Desactiva el reenvío de sesiones gráficas X11. No es necesario en SFTP y mejora la seguridad.

1.2.8 Comprobaciones dentro del contenedor:

Accedemos al contenedor y hacemos una serie de comprobaciones como asegurarnos que el gateway y el dns son los correctos y probamos a hacer ping al dns, que es un contenedor de la subred services y también a un contenedor de la propia red production, por ejemplo al servidor Nginx, por último hacemos otro ping al servidor FTP en development, vemos que el primer ping falla, ya que production no tiene conexión a services, aunque sea la IP del dns, que no llegue el ping no significa que no actúe el DNS sobre production, el resto pings llegan correctamente, todo lo descrito se ve en la siguiente imagen:

```
john@DebianDocker:~/Escritorio/AS/PRACTICAS$ docker exec -it sftp_gateway bash
root@0a11126a2b5d:/# ip route
default via 172.30.0.1 dev eth1
172.18.0.0/16 dev eth2 proto kernel scope link src 172.18.0.9
172.30.0.0/24 dev eth1 proto kernel scope link src 172.30.0.10
172.40.0.0/24 dev eth0 proto kernel scope link src 172.40.0.15
root@0a11126a2b5d:/# cat /etc/resolv.conf
nameserver 172.20.0.6
root@0a11126a2b5d:/# ping 172.20.0.6
PING 172.20.0.6 (172.20.0.6) 56(84) bytes of data.
^C
--- 172.20.0.6 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2038ms

root@0a11126a2b5d:/# ping 172.30.0.5
PING 172.30.0.5 (172.30.0.5) 56(84) bytes of data.
64 bytes from 172.30.0.5: icmp_seq=1 ttl=64 time=4.44 ms
64 bytes from 172.30.0.5: icmp_seq=2 ttl=64 time=0.086 ms
^C
--- 172.30.0.5 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 0.086/2.263/4.441/2.177 ms
root@0a11126a2b5d:/# ping 172.40.0.10
PING 172.40.0.10 (172.40.0.10) 56(84) bytes of data.
64 bytes from 172.40.0.10: icmp_seq=1 ttl=64 time=0.683 ms
64 bytes from 172.40.0.10: icmp_seq=2 ttl=64 time=0.098 ms
^C
--- 172.40.0.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1005ms
rtt min/avg/max/mdev = 0.098/0.390/0.683/0.292 ms
root@0a11126a2b5d:/#
```

Figura 18: Comprobaciones dentro del contenedor sftp_gateway.

1.2.9 Acceso al servidor sftp desde el host:

Para conectarnos a SFTP debemos ejecutar el siguiente comando en nuestra terminal:

```
sftp -p 2224 cliente@localhost
```

Introducimos la contraseña 'password' y se completará la conexión.

Para probar el funcionamiento accedemos al directorio 'sftp_real/ftp_real', aquí podremos mostrar todo el directorio y veremos el contenido del directorio 'publico', podremos acceder a cualquier carpeta unicamente con permisos de lectura, aprovechando que antes probamos subir un archivo en 'SW1' accedemos al subdirectorio e intentaremos descargar el archivo, eliminarlo y subir un archivo nuevo.

```
john@DebianDocker:~/Escritorio/AS/PRACTICAS$ sftp -P 2224 cliente@localhost
cliente@localhost's password:
Connected to localhost.
sftp> cd sftp_data/ftp_real/
sftp> ls
SW1 SW2 SW3 SW4 SW5
sftp> cd SW1
sftp> ls
prueba.txt
sftp> get prueba.txt
Fetching /mnt/ftp_real/SW1/prueba.txt to prueba.txt
prueba.txt                               100% 13    1.1KB/s   00:00
sftp> rm prueba.txt
Removing /mnt/ftp_real/SW1/prueba.txt
remote delete /mnt/ftp_real/SW1/prueba.txt: Permission denied
sftp> put test.txt
Uploading test.txt to /mnt/ftp_real/SW1/test.txt
dest open "/mnt/ftp_real/SW1/test.txt": No such file or directory
sftp> ls -l /mnt/ftp_real/SW1
-rwxr-xr-x  1 root  root      13 May 10 18:03 prueba.txt
sftp>
```

Figura 19: Comprobaciones dentro del servidor sftp.

Como vemos, el usuario puede listar los contenidos incluso descargarlos, pero con los permisos que tiene no le es posible ni subir un archivo ni eliminar los archivo existente.